INF1103

Team Project Framework

Phase 2 - Object-Oriented Refactoring and Extension

1. Overview

In phase 2 of the project your team is expected to take your existing python application from phase 1 and refactor it into a coherent object-oriented system. Beyond this, you will be expected to add one additional meaningful new feature and one advanced feature (described below) while continuing to demonstrate sound software design and development practices. Your class design should not simply wrap existing functions in classes, but it should improve the modularity, maintainability, extensibility and clarity of the system. Your team's real-world problem domain remains the same unless a change is approved by the instructor. The four-manager architecture and the role of AI in the application remain mandatory. There should be no remaining 'procedural' style code left in your application by the end of the project.

2. Project objectives

- Refactor the Phase 1 procedural application into a well designed object-oriented application
- Demonstrate all OOP concepts taught from Weeks 8-11 through meaningful implementation.
- Preserve the four mandatory manager responsibilities and the AI-centered data pipeline.
- Add one useful feature that extends the original application but maintains the object-oriented approach.
- Apply appropriate SOL from SOLID principles (details of SOLID in your lecture notes)
- Use automated tests, Git, Docker, and Continuous Integration (CI) to support reliable development.
- Select one additional feature beyond the lecture notes to add to your project (detailed at the bottom of this brief)

3. Required system architecture

Your system must retain the responsibilities of all four Phase 1 managers. You may reorganise the code into additional modules and classes, but the four responsibilities below must remain identifiable, separated, and functional.

Manager responsibility	Phase 2 expectations
IO Manager	Collect and validate user input; display records, lists, results, and summaries. Keep terminal input/output responsibilities separate from domain logic.
AI Manager	Every record must pass through AI processing. Build prompts, call the API, parse the response, and validate the response schema. Use inheritance or composition where appropriate.
Logic Manager	Apply domain-specific business rules to AI-enriched records. Evaluate records, calculate scores, and route/classify outcomes as appropriate.
Data Manager	Save and load records using CSV, JSON or a database system, support querying/filtering, and handle missing or corrupt files, or connection issues gracefully.

Your design may introduce additional domain classes, services, or helper classes where justified.

4. Mandatory OOP concepts

All concepts listed below must be demonstrated with meaningful implementation in the final deliverable of your application with clear representation in your UML diagram (and likewise UML representation to code!)

Concept	What your implementation must show
Classes and objects	Several purposeful classes with instances used by the running application.
Encapsulation	Access control for attributes and methods is considered and represented through appropriate naming conventions (see lecture notes for details on how this works in python vs other languages)

Abstract classes	At least one meaningful abstract base class that defines shared behaviour and/or abstract operations.
Inheritance	At least one useful subclass relationship that specialises or extends a base class.
Interfaces	At least one interface/contract, implemented using an appropriate Python approach such as an ABC (See lecture notes!) with concrete implementation(s).
Association	At least one meaningful relationship where objects collaborate or refer to one another, without implying whole–part ownership.
Aggregation	At least one meaningful whole–part relationship in which the part can exist independently of the whole.
Composition	At least one meaningful whole–part relationship in which the owning object manages the part's lifecycle or ownership.
Polymorphism	Client code uses a shared interface or base type with different concrete implementations, producing different behaviours without needing type-specific branching for each implementation.

Do not force artificial relationships into your application. Choose examples that make sense for your domain and explain the design. Association, aggregation, and composition must be distinguishable in the design and supported by the implementation and you should be able to justify their use.

5. One new feature

Add at least one new meaningful feature that extends the Phase 1 application. The feature must add useful user or system capability; renaming functions, adding trivial output, or merely converting existing functions into methods does not count as a new feature.

Possible examples (adapt these to your domain):

- Report or summary generation, such as trends, rankings, categories, or an overview of stored records.
- Configurable business rules, thresholds, or routing options.
- Search, filtering, or comparison functionality beyond the original basic query.
- A review or feedback workflow for AI-generated outcomes.
- A history/audit view that compares current results with previously stored records.

Document the feature(s) in your submission. Explain the problem it addresses, how a user accesses it, which classes implement it, and how it is tested.

6. SOLID design principles

Apply SOL from the SOLID principles (you can include more if you wish) where they improve the design. You must explicitly identify and justify at least three principles in your design notes, including Single Responsibility Principle (SRP), Open/Closed Principle (OCP) and Liskov Substitution Principle (LSP). Your implementation must provide evidence; simply defining the principles is insufficient. Refer to your lecture notes for more details.

7. AI and data requirements

- AI remains central to the application: every record being processed must pass through the AI API, and AI output must meaningfully influence the domain logic.
- Prompts must request structured JSON output where appropriate, and responses must be parsed and validated before fields are used.
- Handle API connection failures, timeouts, malformed responses, and invalid response schemas without crashing unnecessarily.
- Persist records to CSV, JSON or a Database and support loading records across separate runs. Handle missing or corrupt data files and / or connection issues with a database gracefully. (Note: using a DB application is not a requirement, CSV/JSON is sufficient)
- Preserve the original domain rules, including the multi-condition rule that uses two or more fields from the AI response, unless an approved change is documented.
- Do not hard-code a successful AI response as a substitute for the required live API processing. Tests may use mock or sample responses so that they run without a live API

8. Testing requirements

- Provide an automated test script or test suite for core logic and important class behaviour.
- Tests must run without a live AI API connection by using suitable sample responses, mocks, or stubs.
- Test the main business rules, scoring/routing behaviour, response validation, and relevant new-feature behaviour.
- Include tests for important failure cases, such as malformed AI output or missing/corrupt persistence files, where applicable.
- Tests must run successfully inside the Docker environment and be runnable by the assessor using documented instructions.

9. DevOps, Git, Docker, and CI

9.1 Git

- Continue development in the team's repository and submit the repository URL and required branch/commit reference.
- Use meaningful, incremental commits that show the progression from Phase 1 to the OOP refactor, new features, tests, and pipeline.
- Commit messages must describe the change. A single monolithic submission commit is insufficient.
- Branches and pull requests are recommended where practical; the commit history should make each team member's contributions and the development process understandable.

9.2 Docker

- Provide a Dockerfile in the repository root and clear build/run instructions.
- The application and automated tests must run in the container without requiring manual dependency installation on the assessor's machine.
- Document any required API credentials or environment variables. Do not commit API keys, passwords, or other secrets to the repository.
- Verify the submission branch builds and runs in Docker before submission.

9.3 Continuous Integration (CI)

A CI workflow is mandatory. For example, use GitHub Actions to run automated tests when code is pushed or a pull request is opened. The workflow must be included in the repository, run successfully for the submitted version, and be documented in the engineering report or README.

10. Required Basic Deliverables

Deliverable	Minimum requirements
1. OOP source code	Runnable Python application, with all four manager responsibilities identifiable and the required OOP concepts implemented.
2. Engineering report (maximum 6 pages)	Include the UML class diagram; an explanation of how each mandatory OOP concept is demonstrated; a brief SOLID justification; a data-flow diagram; and an exception-handling matrix. The CSV output report from TeamAId for team tasks, progress and contributions is submitted as a separate file, not in the report.
3. Automated tests	Tests for core logic, important class behaviour, new features, and relevant failure cases; run without a live API and pass in Docker.
4. Git repository	Repository URL and submission branch/commit; meaningful incremental history; CI workflow included.
5. Docker setup	Dockerfile at repository root and clear build, run, and test instructions.
6. New-feature notes	Identify new feature(s), purpose, the classes involved, and the tests that verify them. These notes may be included in the engineering report.

The engineering report is limited to six pages. Keep explanations concise and make the required evidence easy to locate. The README may contain setup instructions and supplementary details, but it does not replace required report content.

11. Required Advanced Deliverables

To assess the ability for teams to independently explore topics, **any one** of the following should be implemented as part of 'going beyond' the basic module requirements. Some self-exploration and study may be involved in implementing the chosen deliverable in your project. This should be implemented, demonstrated and briefly documented, not merely researched.

If you have anything else you would consider an advanced deliverable, do suggest it to your lab instructor and get clearance before implementation.

11.1 Continuous Deployment (CD)

Teams may implement a suitable deployment or automated release-step if appropriate to their application.

11.2 GUI

A GUI may take many forms, from a web-based interface to a python-based GUI. If you would like to remain in the python environment and build a simple desktop app, PySide6 is a possibility (native looking widgets on all platforms, solid threading support, friendly licensing). If you want to venture beyond python, react, Django are lightweight enough and can communicate well with REST endpoints.

11.3 Database Integration

Currently the default is a CSV / JSON , think about implementing a database, it could be something lightweight and simple enough like MongoDB. The database should have all the usual CRUD operations implemented.

11.4 Threading

Does your UI ever get stuck when your application is doing a more complicated task, would you like to run two processes at once that don't block each other, consider exploring the exciting world of threading!